

The game code checks for a solved puzzle and, when the puzzle is solved, calls the `stop()` method to freeze the time. If the user clicks the Restart button to start a new game, the game calls `start()` to restart the timer object.

CountDownTimer

The `CountDownTimer` class has some subtle differences. Instead of counting up from zero, you initialize a `CountDownTimer` by providing a starting number of seconds, and it counts down from that value. The interface for creating a `CountDownTimer` looks like this:

```
CountDownTimer(nStartingSeconds, stopAtZero=True, nickname=None,
               callBack=None):
```

There is a second optional parameter, `stopAtZero`, that defaults to `True`—which assumes that you want the timer to stop when it reaches zero. You can also optionally specify a function or method as a callback when the timer reaches zero. Lastly, you can supply a nickname to be used if and when a callback is made.

The client calls the `start()` method to begin counting down.

From the client's point of view, the `getTime()`, `getTimeInSeconds()`, `getTimeInHHMMSS()`, and `stop()` methods appear identical to their counterparts in the `CountUpTimer` class.

`CountDownTimer` has an additional method named `ended()`. The application needs to call the `ended()` method every time through its main loop. It returns `False` while the timer is active, but returns `True` when the timer ends (that is, reaches zero).

The countdown version of the Slider Puzzle game's main file is available with the book's resources, at *SliderPuzzles/Main_SliderPuzzleCountDown.py*.

The code is very similar to the previous version that counts up, but this version creates an instance of `CountDownTimer` instead and supplies a set number of seconds it allows to solve the puzzle. It also calls `getTimeInHHMMSS(2)` every frame and updates the time with two decimal digits.